



32. Gradient Descent Methods

The “Brain” of Machine Learning - Learning Iteratively

Previous Section:

 [31. K-Fold Cross Validation](#)

Contents:

[1. What even is Gradient Descent?](#)

[2. Gradient Descent - A Practical Example](#)

[3. Gradient Descent with Python](#)

[4. Gradient Descent - Minimizing Loss Function](#)

[Summary](#)

[References](#)

Every new concept has a way of introducing itself. And this one sounds almost too confident: “**Gradient Descent.**” → It doesn’t explain itself. It doesn’t even try. It just exists. Quietly. Behind almost everything.

But before we touch definitions. Let me tell you how I see it. Imagine a machine. Not just any machine, but one that *learns*.

- It has eyes → it sees data.
- It has ears → it listens to patterns.
- It has hands → it makes predictions.
- Its body? Built from models, algorithms, structures.
- Its heart? → Mathematics.
- Its soul? → Probability.

- Its environment? → Data.
- Its experience? → Evaluation.

And then, there's something more important. Something that tells it: *"You're wrong."* → *"Fix it."* → *"Try again"*. That is the brain. That is **Gradient Descent**.

If Machine Learning is about learning from experience. Then Gradient Descent is how the model *improves from mistakes*. Not instantly. Not magically. But step by step. Again, and again, and again.

Because here's the truth most people don't realize at first: A model doesn't wake up one day and suddenly "understand" data.

- It starts clueless.
- It makes a prediction → it's wrong.
- It adjusts itself → slightly better.
- It tries again → still wrong.
- Adjusts again → closer.

And somewhere along that long sequence of corrections, learning happens.

So if I had to extend the analogy:

- Models are like different personalities
- Data is the world they live in
- Evaluation is feedback from reality

Then Gradient Descent? It's the **thinking process** that connects all of them. The part that says:

┆ *"Based on what just happened... what should I change?"*

And just like humans. Not every model thinks the same way. Some adjust carefully. Some adjust aggressively. Some learn fast but unstable. Some learn slowly, but reliably. Different strategies. Same goal. Get closer to the truth.

1. What even is Gradient Descent?

Now let's slow down and ask the real question: **What exactly is Gradient Descent?**

At first glance, it sounds technical. Almost intimidating.

"An iterative optimization algorithm used to minimize a differentiable function."

→ That's correct. But it doesn't *feel* like understanding. So let's approach it differently.

Imagine you're standing on a mountain. It's foggy. You can't see far ahead. No map. No GPS. No idea where the lowest point is. But you have one advantage. You can feel the ground beneath your feet. You can tell: "This direction goes down." → "That direction goes up".

So what do you do? You take a step downward. Then again. And again. And again. Until eventually, you reach a point where every direction feels like going up. That's when you know: *You've reached a minimum.* → That is Gradient Descent.

Now let's translate that into Machine Learning. Instead of a mountain, we have a function: $f(x)$

And instead of height, we measure something called **loss**. This loss tells us: "*How wrong is the model?*". High loss → very wrong; Low loss → better predictions.

So the goal becomes very simple: **Find the value of x that makes this loss as small as possible.**

But here's the problem. Most functions in Machine Learning are too complex. You can't just solve them with a clean formula. No shortcut. No closed-form answer. So instead, we do what you did on the mountain. We search. Step by step.

Now here's where the "gradient" comes in. At any point on the function, we can compute something called the **gradient**.

Think of it as: *A signal that tells you **which direction goes up the fastest**.* And if we know where it goes up, we can simply go the opposite way. That's where the function goes down.

So the process becomes almost mechanical:

Start somewhere (an initial guess) → Look at the slope (the gradient) →
Move a small step downward → Repeat

Again, and again, and again. Until one day, the movement becomes so small, it barely matters. That's when we stop.

But here's something important. You might think: *"So we always find the best possible minimum?"* Not exactly. A function can have many low points.

- Some are just "locally" low → small valleys.
- One might be the "global" lowest → the deepest valley

Gradient Descent doesn't promise the absolute best. It promises something more realistic:

┆ *"I will find a point that is low enough... to be useful."*

And in most Machine Learning problems? That's more than enough.

So if we step back and simplify everything: Gradient Descent is not magic. It's not even complicated.

It's just: **Start somewhere** → **Look at the slope** → **Move downward** → **Repeat**

And yet this simple idea powers: Linear Regression; Logistic Regression; Neural Networks; Deep Learning → Almost everything that *learns from data*.

Which is why I called it the "brain", Not because it's the most complex part. But because it's the part that keeps asking: ***"How do I become better... from where I am now?"***

In the next section, we'll stop imagining. And actually **watch Gradient Descent in action**.

2. Gradient Descent - A Practical Example

Let's go a little deeper. Because up until now, Gradient Descent sounded almost too perfect. Just "go downhill" and you're done? → Not quite.

A small but important truth: A function is not always a single smooth valley. Sometimes, it looks more like a landscape. Multiple dips. Multiple valleys. Some shallow. Some deep.

So here's the reality: Gradient Descent does **not** try to find *every* minimum. And it doesn't guarantee the **best possible one**. Instead, it does something more practical:

- It finds a **minimum** → Usually the one closest to where it started.

We call these:

- **Local minimum** → a valley nearby
- **Global minimum** → the lowest point overall

And Gradient Descent? It's satisfied with a *good valley (the local minimum)*, not necessarily the deepest one.

How does it actually move? At every step, it asks one simple question: "*Which direction goes up the fastest?*". That answer is the **gradient**. So instead of going up, we go the opposite way. We go downhill. And the process becomes a loop:

- Start from an initial guess x_0
- Compute the gradient
- Take a small step downward
- Repeat. Until nothing changes anymore

Simple idea. But let's make it real. Consider the function:

$$f(x) = \frac{x^2 \cos(x) - x}{10}$$

At first glance, it doesn't look friendly. Nonlinear. Oscillating. Multiple valleys. You can easily tell this by just plotting → Perfect for Gradient Descent.

Step 1: Compute the Gradient

To move downhill, we first need the slope. So we take the derivative. After applying the **product rule** and simplifying, we get:

$$f'(x) = \frac{1}{10} (2x \cos(x) - x^2 \sin(x) - 1)$$

This expression tells us everything:

- Positive value → slope goes up;
- Negative value → slope goes down;
- Near zero → flat region.

This is the "sense of direction" for the algorithm.

Step 2: Update the Position

Now comes the actual movement. Gradient Descent follows this rule:

$$x_{n+1} = x_n - \alpha f'(x_n)$$

Let's translate that:

- $x_n \rightarrow$ where you are now;
- $f'(x_n) \rightarrow$ slope at that point
- $\alpha \rightarrow$ step size (learning rate)

So each step is: Look at slope $\rightarrow$ move opposite $\rightarrow$ adjust position

Step 3: Repeat the Process

Starting from some point x_0 :

- Compute slope
- Move slightly downhill
- Repeat

Over time: Steps become smaller $\rightarrow$ Movement slows down $\rightarrow$ Eventually it stabilizes

That's when we say: A minimum has been reached.

What should you notice? This is where things get interesting.

1. Starting point matters: If you start at different places $\rightarrow$ You may end up in completely different valleys. Example:

- Start at $x_0 = -2 \rightarrow$ one minimum
- Start at $x_0 = 3 \rightarrow$ another minimum

Same function. Different outcome.

2. Learning rate controls everything: That small α term? It decides how you move.

- Too large $\rightarrow$ you overshoot, maybe never settle
- Too small $\rightarrow$ you crawl, painfully slow

So choosing α is not trivial. It's balance.

3. Flat points are tricky: When: $f'(x) \approx 0 \rightarrow$ The algorithm stops.

But here's the catch. That point could be:

- A local minimum

- A local maximum
- A saddle point

Gradient Descent doesn't "know" which one. It just stops moving.

So what did we actually learn? Gradient Descent is:

- Simple → just follow the slope
- Powerful → works on complex functions
- Imperfect → depends on where you start and how you move

And that's what makes it interesting. It's not about perfection. It's about: **Iteratively getting better, until "good enough" becomes powerful.**

3. Gradient Descent with Python

Here's the part I like. Because up until now, we've been doing everything... manually. Derivatives. Updates. Iterations. → It works. But it's not how we actually *build things*.

So let's be honest for a second. You are not going to sit there and compute derivatives by hand every time. That's what tools are for. So instead, we let Python do the heavy lifting.

First, we visualize the function:

```
import matplotlib.pyplot as plt
import numpy as np

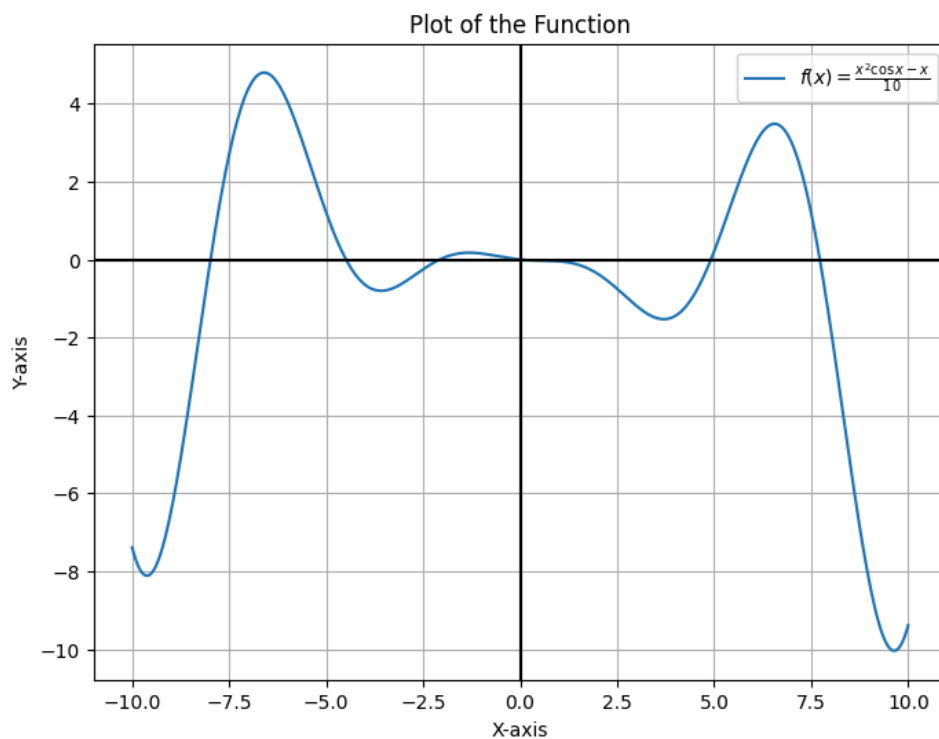
def my_function(x):
    return (x**2 * np.cos(x) - x) / 10

x_values = np.linspace(-10, 10, 400)
y_values = my_function(x_values)

plt.figure(figsize=(8, 6))
plt.plot(x_values, y_values, label=r'$f(x)=\frac{x^2\cos x - x}{10}$')
```

```
# Draw axes through origin
plt.axhline(0, color='black')
plt.axvline(0, color='black')

plt.title('Plot of the Function')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.legend()
plt.show()
```



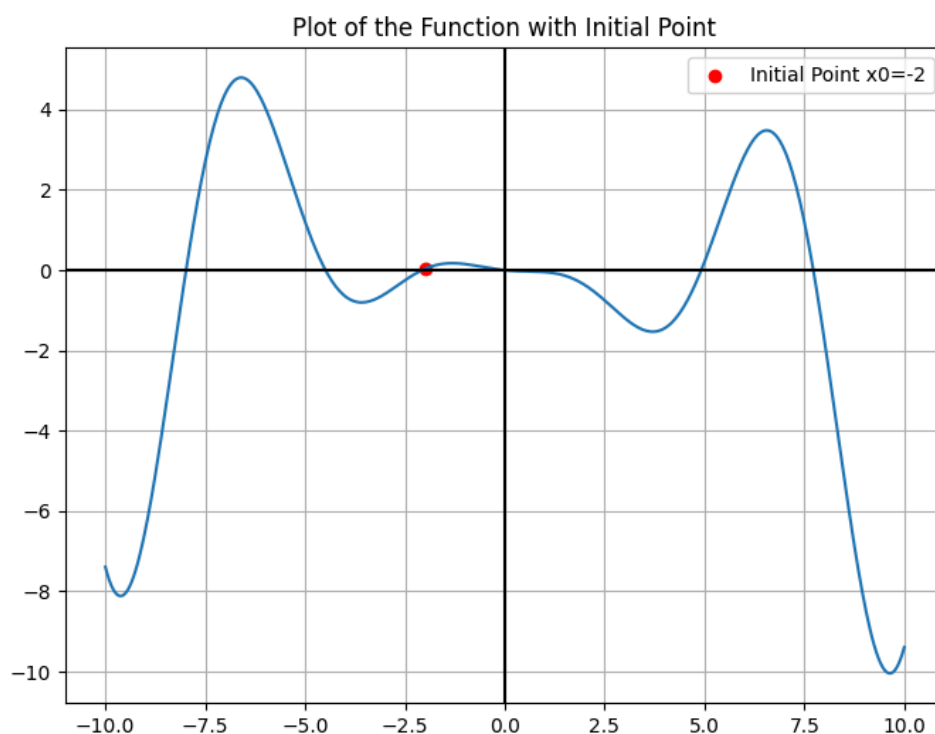
And suddenly, that abstract function? It becomes a landscape. Peaks. Valleys. Slopes. Something you can actually see.

```
# Gradient Descent parameters
x0 = -2

# Plot the x0 point alongside the function
plt.figure(figsize=(8, 6))
plt.plot(x_values, y_values)
```

```
# Draw axes through origin
plt.axhline(0, color='black')
plt.axvline(0, color='black')

plt.scatter(x0, my_function(x0), color='red', label=f'Initial Point x0={x0}')
plt.title('Plot of the Function with Initial Point')
plt.legend()
plt.grid(True)
plt.show()
```



Now pause for a moment. Don't just run it. **Read what it's doing.**

We start at: $x = -2 \rightarrow$ This is your starting position. Your "standing point" on the landscape.

Now let's bring Gradient Descent into it:

```
def gradient_function(x):
    return (2*x*np.cos(x) - x**2*np.sin(x) - 1) / 10
```

```

x0 = -2
learning_rate = 0.1
iterations = 20

# Run Gradient Descent
x_gd = [x0]

for _ in range(iterations):
    x_new = x_gd[-1] - learning_rate * gradient_function(x_gd[-1])
    x_gd.append(x_new)

x_gd = np.array(x_gd)
y_gd = my_function(x_gd)

# Print the results
for i in range(len(x_gd)):
    print(f"At x = {x_gd[i]}, y = {y_gd[i]}")

```

(You will obtain multiple output when you run this code → This depends on your chosen number of iterations)

And then, step by step: Compute the slope → Move slightly downhill → Record the new position → Again, and again, for 20 iterations.

So what do you get? A sequence of values: $x_0, x_1, x_2, \dots, x_{20}$ → Each one lower than before (hopefully). And each x has a corresponding y .

Now here's the interesting part. If you look at the final output, you might see something like:

$$x \approx -3.20112 \rightarrow y \approx -0.70279$$

```

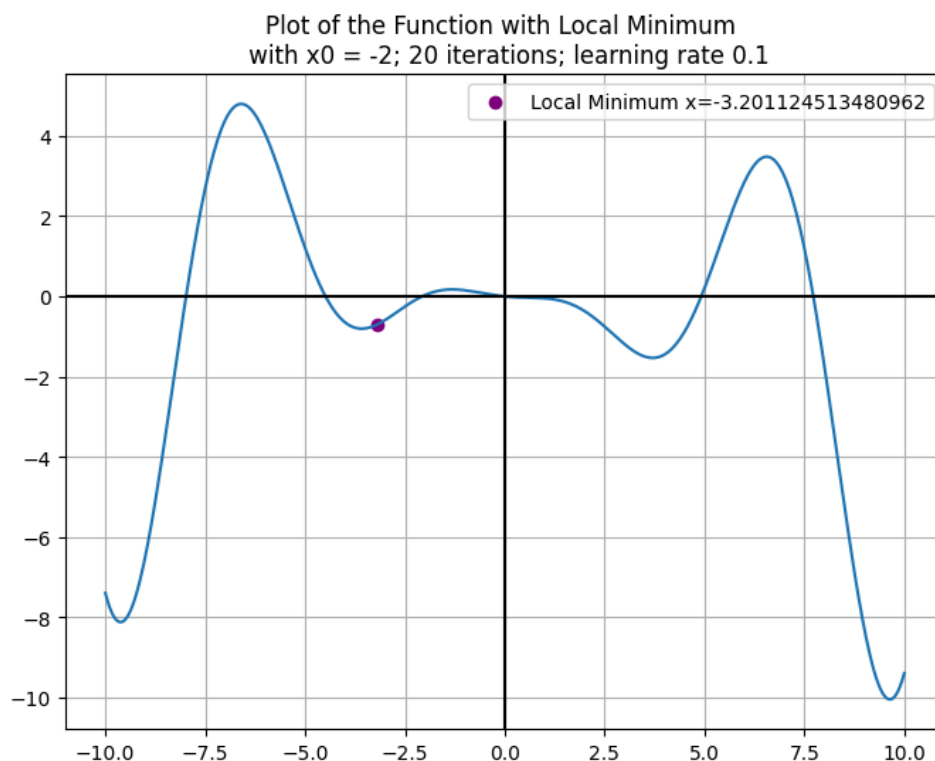
# Plot the local minimum point found
plt.figure(figsize=(8, 6))
plt.plot(x_values, y_values)

# Draw axes through origin
plt.axhline(0, color='black')

```

```
plt.axvline(0, color='black')

plt.scatter(x_gd[-1], my_function(x_gd[-1]), color='purple', label=f'Local Minimum x={x_gd[-1]}')
plt.title(f'Plot of the Function with Local Minimum \n with x0 = {x0}; {iterations} iterations; learning rate {learning_rate}')
plt.legend()
plt.grid(True)
plt.show()
```



So naturally, the question appears: *“Did we just find one of the local minimums?”* And the answer is: **Maybe.**

More precisely, we likely found **a local minimum**. A point where:

- Moving left → goes up; Moving right → goes up. So the algorithm stops.

But here’s the catch. Does this mean: This is the **best possible value**? The **deepest valley** in the entire function? → **Absolutely not.**

Because remember the landscape. There could be:

- A deeper valley somewhere else
- A lower point we never reached
- A better solution hidden far away

And Gradient Descent? It never explored those. It just followed the path from where it started.

So what did we really learn here?

- Gradient Descent is **path-dependent**
- Your starting point matters
- Your final answer is not always the best, just *one of the good ones*

And this is where things become real. Because in Machine Learning: We don't chase perfection. We chase **good enough solutions that work**

And Gradient Descent? It's not trying to be perfect. It's just trying to get better. Step by step. Until it can't anymore.

4. Gradient Descent - Minimizing Loss Function

Let's step back for a moment. Because so far, Gradient Descent feels like a nice idea. A method. A process. A way to walk downhill.

But here's the real question: *Where does it actually matter?*

The answer is: **Almost everywhere**. Gradient Descent is not tied to one model. It's not "for regression" or "for classification". It's something deeper. It's the **engine** that makes models learn.

Let's see how that plays out. **Where Gradient Descent actually lives?**

1. Linear Regression — "Fit the best line"

Start with something simple. You have data. Points scattered in space. And you want a line that fits them. So you write: $\hat{y} = a + bx \rightarrow$ A straight line.

But how do you know if it's good? You measure the error.

$$L = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

This is the **Mean Squared Error (MSE)**.

Now here's where Gradient Descent enters. It asks: *"How should I adjust a and b to reduce this error?"*. And step by step:

- Compute how wrong the model is
- Measure how changing a, b affects the error
- Adjust them slightly
- Repeat

So instead of solving everything at once. The model *learns the line* gradually.

Especially useful when:

- Data is large
- Features are many
- Exact solutions are too expensive

2. Logistic Regression — "Learn probabilities"

Now we move to classification. Instead of predicting numbers, we predict probabilities. So we pass values through a sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

And instead of MSE, we use a different loss:

$$L = -\frac{1}{n} \sum [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

This is **cross-entropy loss**.

Now the question becomes: *"How do we adjust the weights to improve these probabilities?"*

Again, Gradient Descent answers.

- Compute gradients of the loss
- Adjust weights
- Improve predictions

And this is important. Because now the function is **nonlinear**. You can't just "solve it nicely". But Gradient Descent? It doesn't care. It just keeps stepping

until things improve.

3. Softmax Regression — “Choose between many classes”

Now imagine more than two classes. Not just: Yes / No → But: Cat / Dog / Bird / Horse / ...

So we extend the idea. We use Softmax:

$$\hat{p}_{i,k} = \frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}}$$

And the loss becomes:

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{i,k} \log(\hat{p}_{i,k})$$

Now things get more complex. Multiple classes. Multiple weight vectors. But the idea? Exactly the same.

Gradient Descent:

- Computes gradients for each class
- Adjusts all weights
- Improves the probability distribution

And step by step, the model learns how to separate multiple categories.

So what's the big picture? Linear → Nonlinear; Binary → Multiclass.

Different models. Different goals. But underneath? The same mechanism.

Gradient Descent is not “part of the model”. It's the thing that *makes the model learn at all*.

It doesn't care if you're fitting a line, or building a deep neural network. It just keeps asking: “How wrong am I?” → “How should I change?”

And by repeating that question again, and again, and again. Intelligence starts to emerge. Not from formulas. But from **iteration**.

Summary

Gradient Descent is hilarious. It's not flashy. It's not the model you brag about. It's not the "main character" in most discussions. But take it away? Nothing works.

It's like that one person in a group project, Not the leader. Not the presenter. Not the one getting all the credit. But the one quietly doing the work. Fixing mistakes. Adjusting things. Making sure everything actually improves.

And that's exactly what Gradient Descent does. It doesn't define the model. It doesn't make the final decision. But it's the one constantly asking: *"Can we do better?"* → *"Let's try again."*

With that said, Gradient Descent is not the one that stands out. But it's the one you absolutely cannot remove. Because without it? The model doesn't learn.

References

<https://www.geeksforgeeks.org/machine-learning/gradient-descent-algorithm-and-its-variants/>

<https://www.ibm.com/think/topics/gradient-descent>

<https://www.youtube.com/watch?v=sDv4f4s2SB8&themeRefresh=1>

<https://www.youtube.com/watch?v=qg4PchTECck>

Next Section:

 [33. Convex Optimization Problems](#)